

Audit 3.0 Engineering Portfolio

Sprint Review & Architectural Hardening Baseline Session

1. Executive Sprint Summary

System Entry State

The codebase was relying on a corrupted, local 1-line text error file hidden under a .gguf extension. The runtime configurations were configured for unthrottled maximum GPU usage (`n_gpu_layers=-1`) inside an active execution worker loop, resulting in critical VRAM overflow thrashing and total system kernel locks under asynchronous processing.

System Exit State

The system has been completely hardened. A true, verified 5.76GB Gemma-2 model binary is deployed safely. The core **Enterprise Pro** batch audit routine is completely isolated and untampered, running perfectly. A network-hardened dev assistant layer (ask and chat commands) has been established with built-in client timeout properties to protect your terminal from drops on weak Wi-Fi networks.

2. Step-by-Step Evolution & Technical 'Why'

Action

Swapped deprecated `huggingface-cli download` for modern `hf download` and forced raw storage location streaming parameters via `--local-dir`.

System Impact

Replaced a broken 1-byte text error response page with a genuine 5.76GB neural network weight layer collection map matching standard Linux file signatures.

Core Justification

Bypassed local system memory loaders crashing on unreadable file schemas. `llama.cpp` requires specific layout allocations to initialize safely.

Action

Configured `n_gpu_layers=20`, `n_ctx=2048`, and capped processing cycles down to `n_threads=4`.

System Impact

Split model processing weights. 20 layers load onto your dedicated 6GB RTX 4060 VRAM, leaving breathing room for your Linux UI, while the remaining blocks sit inside system RAM.

Core Justification

Prevents destructive memory thrashing. Setting unconstrained layers forces Linux to page data across slow channels, completely freezing hardware interfaces.

Action

Enforced a strict `asyncio.Semaphore(1)` serialization constraint specifically when the `--local` argument flag is raised.

System Impact

Intercepts the processing loop. Cloud routines process files concurrently at high velocity, but local runs process files sequentially, one-by-one.

Core Justification

Thread Safety. The underlying C++ execution graphs inside `llama_cpp` are thread-unsafe; simultaneous shared queries cause instant memory pointer collisions and **Segmentation Faults**.

Action

Upgraded the `src/cli.py` interface using `typer`, adding `ask` and `chat` subcommands using `HttpClientConfig` and `HttpOptions(timeout=15.0)`.

System Impact

Exposes an on-demand, streaming conversational connection to high-tier reasoning engines (`gemini-2.5-pro`) completely isolated from your automated work data sweeps.

Core Justification

Removes browser dependency. The 15-second client timeout ensures that if your local Wi-Fi drops packets, the terminal catches the error gracefully instead of hanging indefinitely.

 **Architectural Trade-Off: Local Processing Speed vs. Cost Efficiency**

During local offline execution, your hardware dropped to processing **3.49 tokens per second**. While this provides 100% data privacy and zero API costs, it requires roughly 1.5 minutes of continuous high-power hardware rendering per file. The cloud path represents a massive boost in velocity for heavy business operations while you are working on a slower network setup.

3. The 'Under-the-Hood' Deep Dive: Prompt Token Inversion

The most fascinating technical challenge handled this session was resolving the **System role not supported** error thrown by Google's native **Gemma-2 architecture** tokenizer logic.

Standard model configurations separate prompts into distinct structural abstractions: **system** instructions (global rules) and **user** turns (the dynamic data). However, Gemma's compiled model metadata contains a strict Jinja layout filter rule: `{% if messages[0]['role'] == 'system' %}{{ raise_exception('System role not supported') }}{% endif %}`.

To bypass this hard structural boundary without breaking our application layer, we implemented an architectural layout wrapper. The application intercepts local runs, completely strips out the system message dictionary object, and merges your deep compliance audit rules right into the **user** content string payload block as an explicit block header. This satisfies the model's strict requirement for alternating conversation turns while successfully enforcing identical quality control bounds over the local model's generation window.

4. Architectural Guardrails for Next Session

**The Enterprise Audit Rule:**

Never modify or edit the core logic inside the main

`run_audit_engine` pipeline block without explicitly keeping its structured output mappings intact for production security.

**Model Concurrency Limits:**

Never initialize a local

`Llama` instance inside concurrent worker tasks or allow multiple async tasks to call it simultaneously. Local execution blocks must always be choked to a maximum concurrency of 1.

**Indentation Formatting Controls:**

tabs are completely banned from edits inside this repository. All indentation configurations must use strict 4-space increments to keep the python interpreter completely happy.

**API Client Modality:**

When interacting with conversational stream handles via the updated GenAI SDK, use

`chat.send_message_stream(user_input)`

to maintain stable, progressive text rendering.

html>